

Program 1

Create collection reg and solve the query using the concept of regex.

```
db.reg.insertMany([ {name:"test",age:22},{name:"Test",age:21}, {name:"tesT",age:20}, {name:"test123",age:23}, {name:"123test",age:24}, {name:"123test",age:25}, {name:"123",age:26} ])
```

case 1: Find all names where “test” occurs as a substring or as a separate word.

case 2: To have a **case-insensitive** matching we can append **/i** identifier after RE

case 3: To match a string beginning with “test” only.

case 4: To end with “test” only.

case 5: To end with digit only.

case 6: To start with digit only.

case 7: To accept only digits, nothing else. Not even blank.

case 8: To accept only with empty string also.

case 9: To match having a name of 3-10 letters only.

Program 2

Create collection student and solve the query using the concept of regex.

_id	name	age	city	marks	status
1	Amit	18	Ahmedabad	75	A
2	Ankit123	20	Surat	82	A
3	Riya	21	Rajkot	68	B
4	TestUser	22	Vadodara	90	A
5	UserTest	19	Ahmedabad	71	B
6	Testing	20	Surat	84	A
7	ABC123	18	Rajkot	65	B
8	123ABC	23	Ahmedabad	88	A
9	98765	22	Bhavnagar	59	C
10	Anjali	20	Surat	80	A

1. Display students whose name contains "Test".
2. Display students whose name contains "test" (case-insensitive).
3. Display students whose name starts with "A".
4. Display students whose name ends with "Test".
5. Display students whose name starts with a digit.
6. Display students whose name ends with a digit.
7. Display students whose name contains only digits.
8. Display students whose name contains only letters.
9. Display students whose name starts with a letter and ends with a digit.
10. Display students whose name starts with "A" or "R".
11. Display students whose name contains **at least one digit**.
12. Display students whose name starts with **two uppercase letters**.

13. Display students whose name starts with a letter and has **exactly three digits at the end**.